

MLOps Exam Project Report

From Notebook Experimentation To Reproducible ML Workflows

End-To-End MLOps Workflow

A structured implementation report for a machine learning text classification system.

Code Organization Logging Reproducibility Automation

Student: Ayoub Moutik

Course: DevOps / MLOps

Project: MLOpsFull and Monitoring-ML

Date: May 13, 2026

Contents

Abstract	2
1 Project Context And Objective	2
1.1 Context	2
1.2 Objective	2
2 Part 1: Moving From Notebooks To Scripts	2
2.1 Purpose	2
2.2 Existing Notebook	2
2.3 Script-Based Architecture	3
2.4 Module Responsibilities	3
2.5 Notebook-To-Script Mapping	4
2.6 Benefits For MLOps	4
3 Part 2: Logging For ML Systems	5
3.1 Purpose	5
3.2 Logging Configuration In The Project	5
3.3 Where Logs Are Used	6
3.4 Why Logging Matters For This Project	6
3.5 Production Extension	6
4 Part 3: Reproducibility	7
4.1 Purpose	7
4.2 Code Versioning With Git	7
4.3 Dependency Versioning	7
4.4 Data Versioning	8
4.5 Model And Experiment Versioning With MLflow	8
4.6 Checkpointing	9
4.7 Random Seeds	9
4.8 Reproducibility Summary	9
5 Part 4: CI/CD With Jenkins	9

Abstract

This report presents the design and implementation of an MLOps workflow for a machine learning text classification project. The system is organized around a modular Python codebase that supports data processing, model training, evaluation, prediction, and serving. The workflow emphasizes maintainability, traceability, and reproducibility, which are essential requirements for reliable machine learning systems.

The report begins by describing the project architecture and the transition from notebook-based experimentation to script-based engineering. It then presents the logging strategy used to capture execution details and operational events. Finally, it analyzes the reproducibility mechanisms provided by Git, dependency management, MLflow tracking, model checkpointing, dataset organization, and deterministic utilities.

1 Project Context And Objective

1.1 Context

The project material is composed of five course PDFs and two repositories:

- **MLOpsFull**: the main machine learning application.
- **Monitoring-ML**: a monitoring notebook and README covering model performance, drift detection, and online monitoring.

The main application is a text classification system inspired by the Made With ML project. It classifies machine learning project descriptions into tags such as computer vision, natural language processing, graph learning, MLOps, reinforcement learning, and other categories.

1.2 Objective

The objective is to demonstrate an end-to-end MLOps workflow. The project includes the foundational engineering practices required before automation and monitoring can be introduced reliably:

1. moving from notebooks to scripts;
2. logging for ML systems;
3. reproducibility through versioned code, data, models, and dependencies.

2 Part 1: Moving From Notebooks To Scripts

2.1 Purpose

Notebooks are excellent for exploration because they allow interactive execution, visual inspection, and fast experimentation. However, they are not ideal as the final structure of a production ML project. A notebook can hide execution order, retain variables in memory, and make testing or automation difficult.

Scripts provide a cleaner engineering structure. They make workloads easier to test, execute from a terminal, reuse in CI/CD, and deploy as services. This is the first major step from data science experimentation toward MLOps.

2.2 Existing Notebook

The original notebook is:

```
MLOpsFull/mlops_Mine - Copy.ipynb
```

Its main sections include setup, data ingestion, preprocessing, training, experiment tracking, evaluation, interpretability, prediction, and serving. This is a typical research-to-development notebook: it contains the full workflow, but the logic is not yet presented as independent production components.

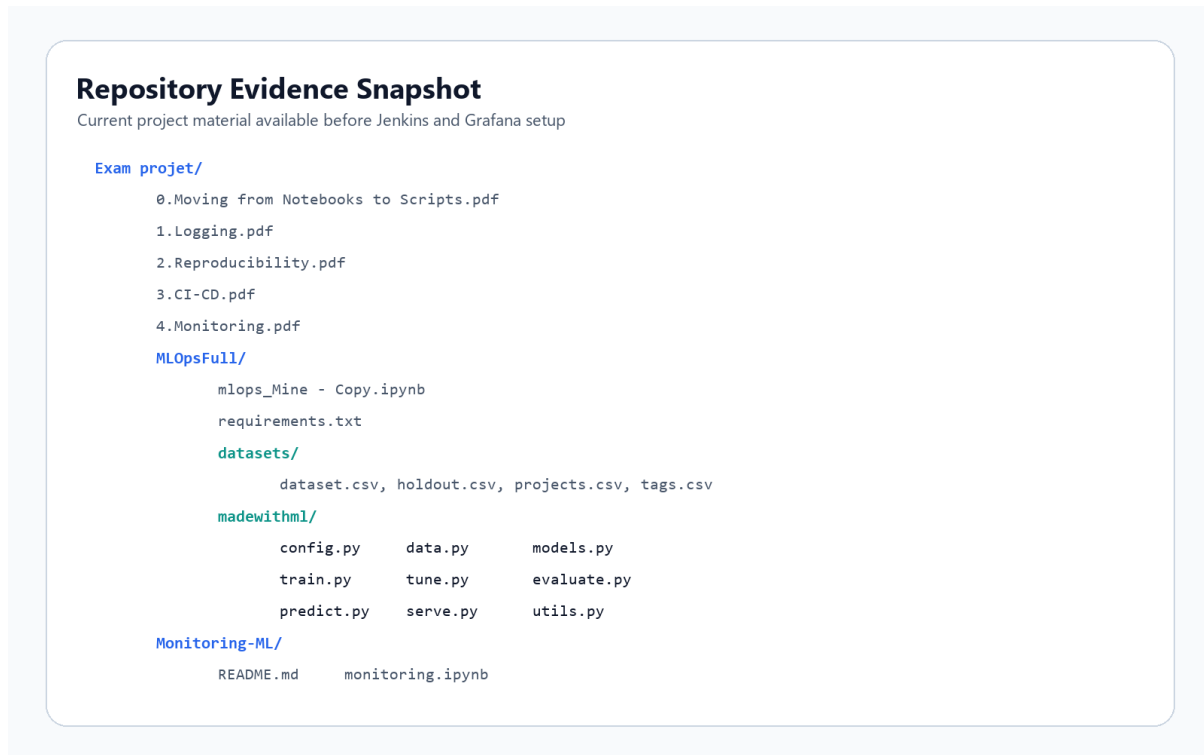


Figure 1: Current project material available in the workspace.

2.3 Script-Based Architecture

The notebook workflow has been separated into modules under `MLOpsFull/madewithml`. Each file owns a specific responsibility, which improves maintainability and makes the project easier to automate.

2.4 Module Responsibilities

Module	Responsibility
<code>config.py</code>	Defines shared project configuration, logging paths, MLflow tracking URI, and constants such as stopwords.
<code>data.py</code>	Loads datasets, performs stratified splitting, cleans text, tokenizes input, and applies preprocessing.
<code>models.py</code>	Defines the PyTorch model architecture used for text classification.
<code>train.py</code>	Contains the Ray Train workload, training loop, checkpointing, and MLflow callback integration.
<code>tune.py</code>	Contains the hyperparameter tuning workload using Ray Tune and HyperOpt.
<code>evaluate.py</code>	Computes overall, per-class, and slice-level metrics.
<code>predict.py</code>	Loads model checkpoints and performs prediction or probability estimation.
<code>serve.py</code>	Defines the FastAPI application and Ray Serve deployment.
<code>utils.py</code>	Provides shared utility functions such as seed setting, batching, and dictionary saving.

Moving from Notebooks to Scripts

Objectif: How to migrate and organize code from our notebook to Python scripts.

Intuition: Notebooks have been great so far for development. They're interactive, stateful (don't have to rerun code), and allow us to visualize outputs. However, when we want to develop a quality codebase, we need to move to scripts. Here are some reasons why:

- **stateless:** when we run code in a notebook, it's automatically saved to the global state (memory). This is great for experimentation because code and variables will be readily available across different cells. However, this can be very problematic as well because there can be hidden states that we're not aware of. Scripts, on the other hand, are stateless and we have to explicitly pass variables to functions and classes.
- **linear:** in notebooks, the order in which we execute cells matter. This can be problematic because we can easily execute cells out of order. Scripts, on the other hand, are linear and we have to explicitly execute code for each workload.
- **testing:** it's significantly easier to compose and run tests on scripts, as opposed to Jupyter notebooks. This is crucial for ensuring that we have quality code that works as expected.

Setup: We already have all the scripts provided in our repository so let's discuss how this was all organized.

README

It's always a good idea to start organizing our scripts with a `README.md` file. This is where we can organize all the necessary instructions to walk through our codebase. Our README has information on how to set up our environment, how to run our scripts, etc.

The content of the `README.md` file is what everyone will see when they visit your repository on GitHub. So, it's a good idea to keep it updated with the latest information.

Figure 2: Course slide evidence: the first topic is moving from notebooks to scripts.

2.5 Notebook-To-Script Mapping

2.6 Benefits For MLOps

This structure is important because Jenkins, tests, model training jobs, and deployment commands all need scriptable entry points. A CI/CD server cannot reliably execute arbitrary notebook cells in a human-controlled order. It needs deterministic commands and modules.

The current organization creates a foundation for:

- command-line training;
- automated evaluation;
- experiment tracking;
- API deployment;
- future Jenkins automation.

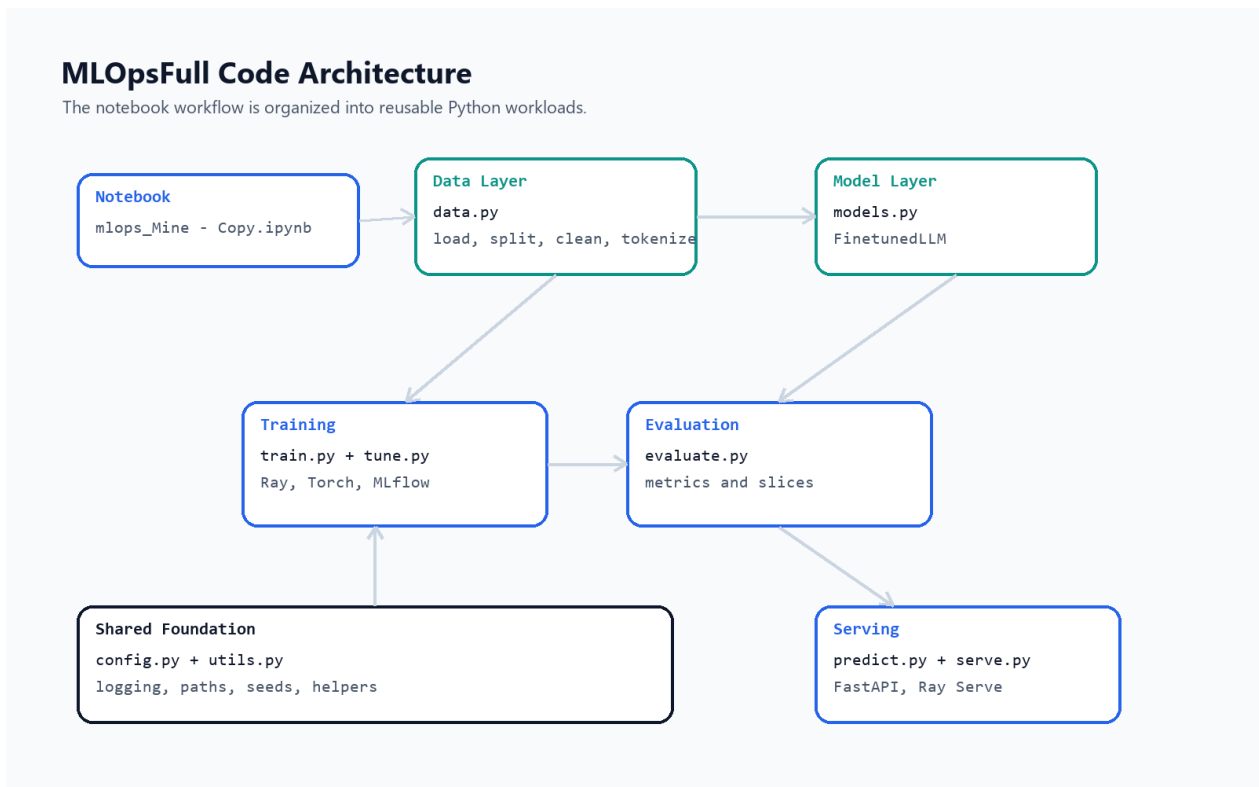


Figure 3: Script organization of the MLOpsFull project.

3 Part 2: Logging For ML Systems

3.1 Purpose

Logging records what happens inside the application during execution. In ML systems, logging is essential because failures can come from code, data, training configuration, model artifacts, infrastructure, or serving inputs.

Compared with `print`, logging is more suitable because it supports severity levels, multiple outputs, timestamps, persistent files, and consistent formatting.

3.2 Logging Configuration In The Project

The logging setup is defined in:

MLOpsFull/madewithml/config.py

The project creates a `logs` directory and defines several handlers:

- a console handler for terminal output;
- an `info.log` rotating file handler for informative logs;
- an `error.log` rotating file handler for errors.

Key implementation pattern:

```

LOGS_DIR = Path(ROOT_DIR, "logs")
LOGS_DIR.mkdir(parents=True, exist_ok=True)

logging_config = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "console": {...},
    },
}

```

Notebook Area	Script Destination
Data loading and preprocessing	madewithml/data.py
Model definition	madewithml/models.py
Training loop	madewithml/train.py
Hyperparameter search	madewithml/tune.py
Evaluation	madewithml/evaluate.py
Prediction	madewithml/predict.py
Serving	madewithml/serve.py
Shared configuration	madewithml/config.py
Shared helpers	madewithml/utils.py

Table 2: Mapping from notebook sections to production-oriented scripts.

```

    "info": {...},
    "error": {...},
  },
  "root": {
    "handlers": ["console", "info", "error"],
    "level": logging.INFO,
  },
}

logging.config.dictConfig(logging_config)
logger = logging.getLogger()

```

3.3 Where Logs Are Used

The project uses `logger.info(...)` to record important workflow outputs. For example:

- training logs the timestamp, run ID, parameters, and metric history;
- evaluation logs overall metrics, per-class metrics, and slice metrics;
- prediction logs model prediction results.

3.4 Why Logging Matters For This Project

In this project, logging supports three important MLOps needs:

1. **Debugging:** errors and warnings can be inspected after a failed run.
2. **Traceability:** training and evaluation outputs can be connected to model runs.
3. **Monitoring readiness:** logs can later be exported to tools such as Loki, Elasticsearch, or cloud logging services and visualized in Grafana.

3.5 Production Extension

For the final version, logs can be connected to the monitoring stack. A practical production-style design would be:

Application logs -> Loki or Elasticsearch -> Grafana dashboards

This will allow the report to include screenshots of API errors, model prediction logs, and system events once the serving and monitoring stages are implemented.

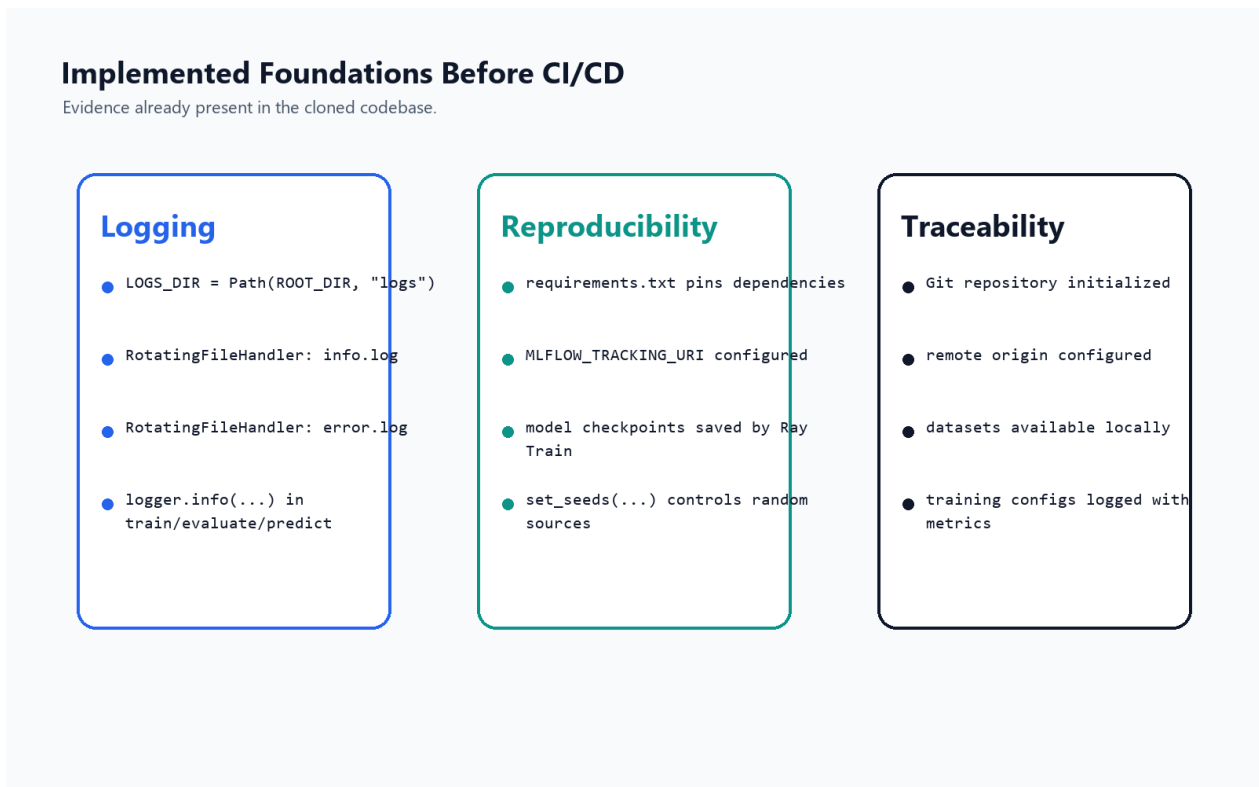


Figure 4: Evidence of implemented logging, reproducibility, and traceability foundations.

4 Part 3: Reproducibility

4.1 Purpose

Reproducibility means being able to recreate a previous result. In machine learning, this is difficult because results depend on code, data, dependencies, random seeds, training parameters, model artifacts, and hardware.

A reproducible ML project should answer:

- Which code version produced this model?
- Which dataset was used?
- Which dependencies were installed?
- Which parameters were used?
- Which metrics were obtained?
- Where is the model artifact stored?

4.2 Code Versioning With Git

The repositories are Git repositories and are connected to a remote origin. Git provides history, collaboration, rollback, and traceability.

```
origin https://github.com/AyoubMoutik/mlops-project.git
```

In the final report, screenshots from GitHub or Jenkins checkout logs can be added to strengthen this section.

4.3 Dependency Versioning

The project includes:


```
MLOpsFull/requirements.txt
```

This file pins or constrains many important dependencies, including:

- mlflow
- ray
- torch
- transformers
- scikit-learn
- fastapi
- pytest
- great-expectations

This is necessary because dependency changes can alter model behavior or break the pipeline.

4.4 Data Versioning

For this educational project, the datasets are stored directly in the repository:

```
MLOpsFull/datasets/dataset.csv
MLOpsFull/datasets/holdout.csv
MLOpsFull/datasets/projects.csv
MLOpsFull/datasets/tags.csv
```

This is acceptable because the datasets are small. In production, larger datasets should not usually be stored directly in Git. Better options include:

- DVC;
- Git LFS;
- S3 or another object store;
- Snowflake or another warehouse;
- versioned data lake storage.

4.5 Model And Experiment Versioning With MLflow

The project configures MLflow tracking in `config.py`. The model registry path is derived from the configured storage directory, and MLflow is set using a file URI.

```
MODEL_REGISTRY = Path(f"{EFS_DIR}/mlflow")
MLFLOW_TRACKING_URI = "file://" + str(MODEL_REGISTRY.absolute())
mlflow.set_tracking_uri(MLFLOW_TRACKING_URI)
```

During training, the project uses an MLflow callback:

```
mlflow_callback = MLflowLoggerCallback(
    tracking_uri=MLFLOW_TRACKING_URI,
    experiment_name=experiment_name,
    save_artifact=True,
)
```

This supports experiment tracking by storing parameters, metrics, checkpoints, and artifacts.

4.6 Checkpointing

The training workflow creates checkpoints and keeps the best checkpoint based on validation loss:

```
checkpoint_config = CheckpointConfig(
    num_to_keep=1,
    checkpoint_score_attribute="val_loss",
    checkpoint_score_order="min",
)
```

This matters because the best model can be loaded later for prediction, evaluation, or deployment.

4.7 Random Seeds

The project contains a utility function for setting random seeds. This reduces randomness across NumPy, Python, PyTorch, CUDA, and hashing behavior.

```
def set_seeds(seed: int = 42):
    np.random.seed(seed)
    random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
```

This does not guarantee perfect reproducibility on every machine, but it is an important baseline.

4.8 Reproducibility Summary

Requirement	Current Project Evidence
Code versioning	Git repository with remote origin configured.
Dependency versioning	<code>requirements.txt</code> .
Data availability	CSV datasets in <code>MLOpsFull/datasets</code> .
Experiment tracking	MLflow tracking URI and Ray MLflow callback.
Model artifact tracking	Ray checkpoints and MLflow artifacts.
Configuration tracking	Training parameters logged with results.
Randomness control	<code>set_seeds</code> utility.

Table 3: Current reproducibility mechanisms in the project.

5 Part 4: CI/CD With Jenkins